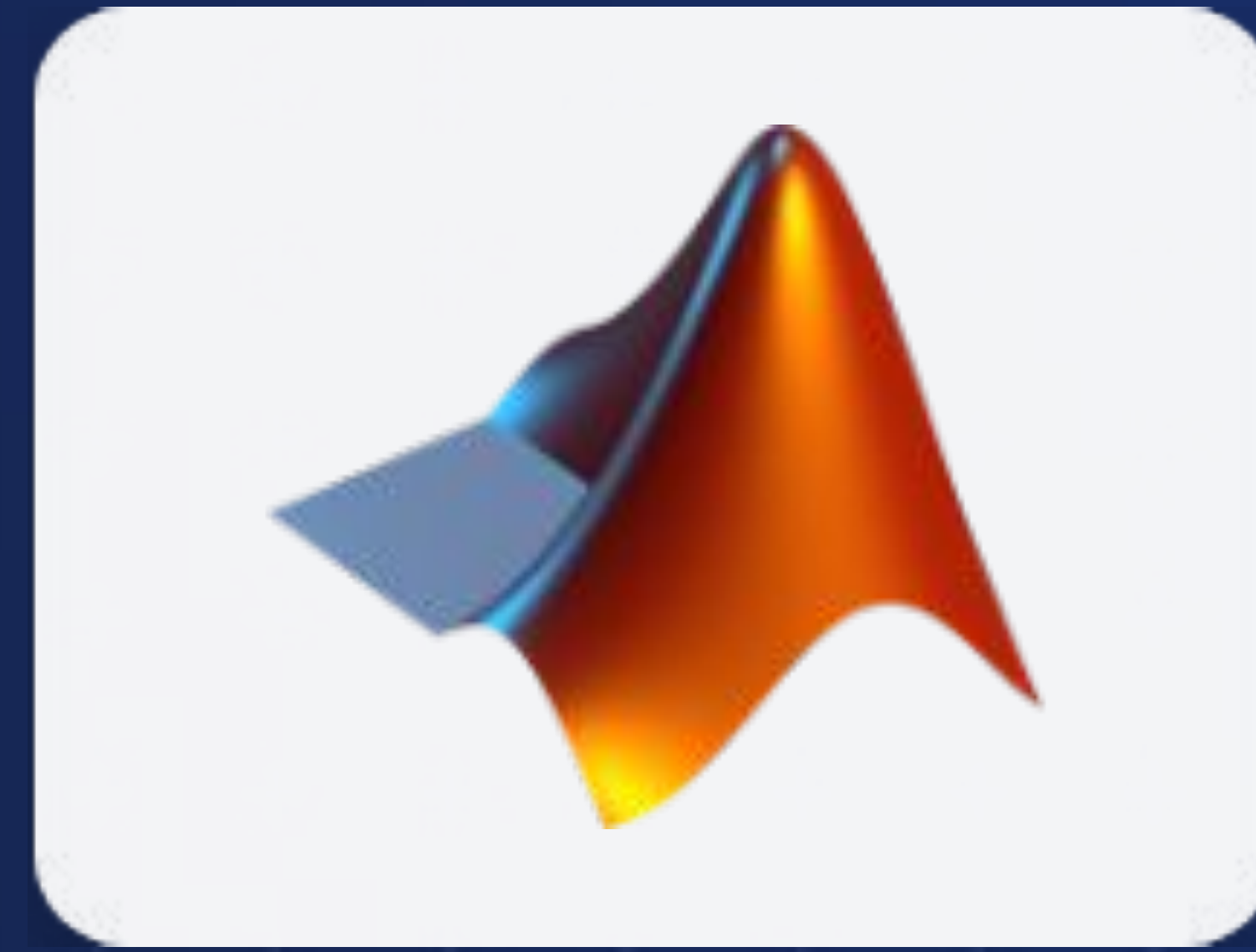




Diffusion-Reaction Analysis

Solving differential equations using Finite Difference & Cubic Splines in **MATLAB**

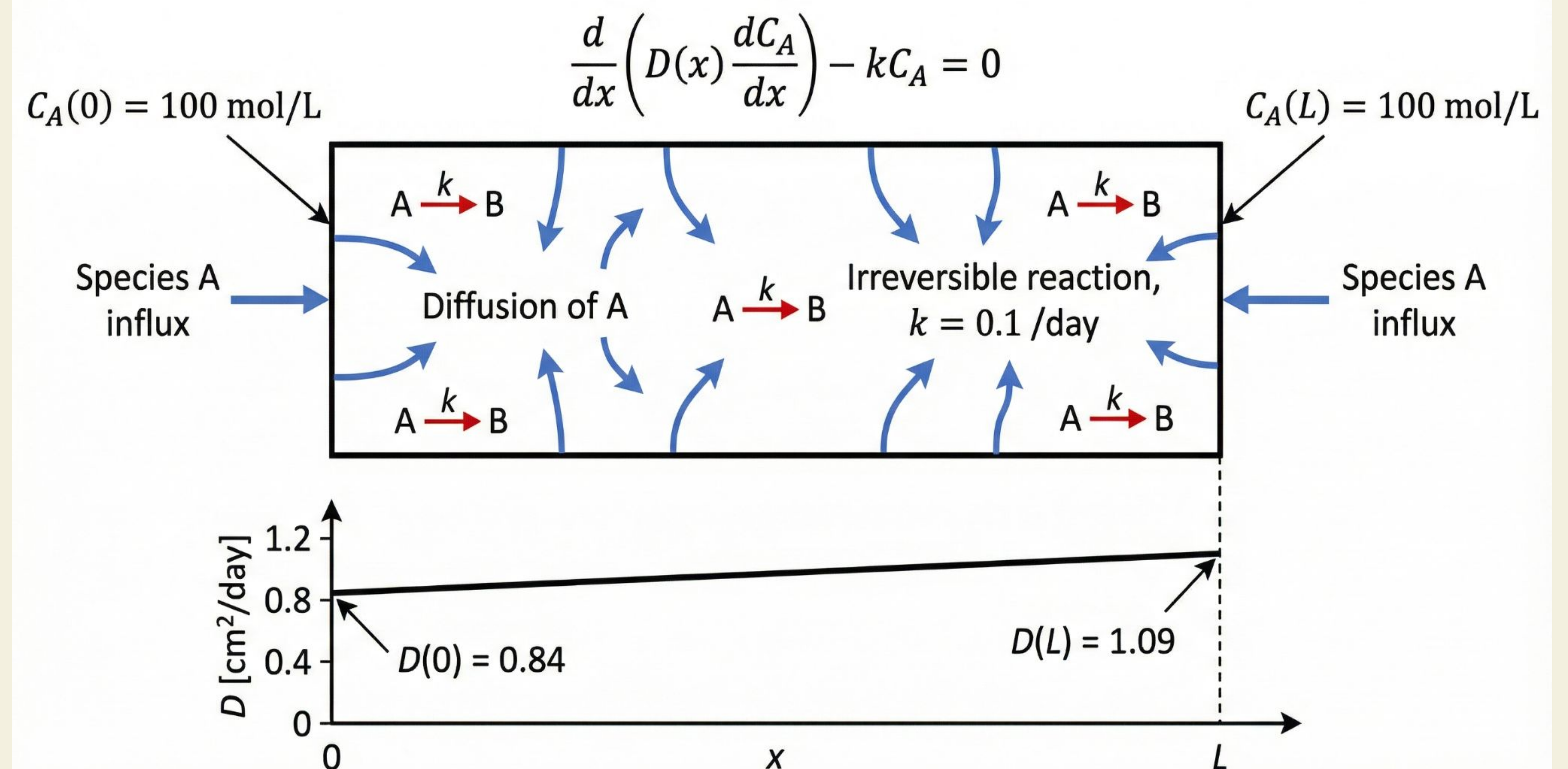
The Physical Problem

Species A diffuses through a liquid layer ($L=5\text{cm}$) while reacting to form B.

- **How Diffusion Varies:**
 $D(x) = 0.05x + 0.84$
- **Rate Constant (k):** 0.1 day^{-1}
- **Boundary Conditions:**
 $C(0) = 100, C(5) = 100$
- **Step Size (Δx):** 0.05 cm

$$\frac{d}{dx} D(x) \frac{dC}{dx} - kC = 0$$

Steady-State Diffusion and Reaction of Species A with Variable Diffusivity



Numerical Methodology



Finite Difference (FDM)

Discretizes the domain into a grid. Converts the continuous Ordinary Differential Equation (ODE) into a system of algebraic linear equations.



Natural Cubic Spline

Interpolation technique. Takes the discrete points from FDM and creates a smooth, continuous piecewise function to connect them.

FDM Formulation

Substituting the finite difference terms yields the general algebraic equation for every node i .

$$D \frac{d^2 C}{dx^2} + \frac{dD}{dx} \frac{dC}{dx} - kC = 0$$

Coefficients:

$$\alpha_i = D_i / \Delta x^2 - D'_i / 2\Delta x$$

$$\beta_i = -2D_i / \Delta x^2 - k$$

$$\gamma_i = D_i / \Delta x^2 + D'_i / 2\Delta x$$

Then apply Central Difference approximations for derivatives:

$$C'' \approx \frac{C_{i+1} - 2C_i + C_{i-1}}{\Delta x^2}$$

$$C' \approx \frac{C_{i+1} - C_{i-1}}{2\Delta x}$$

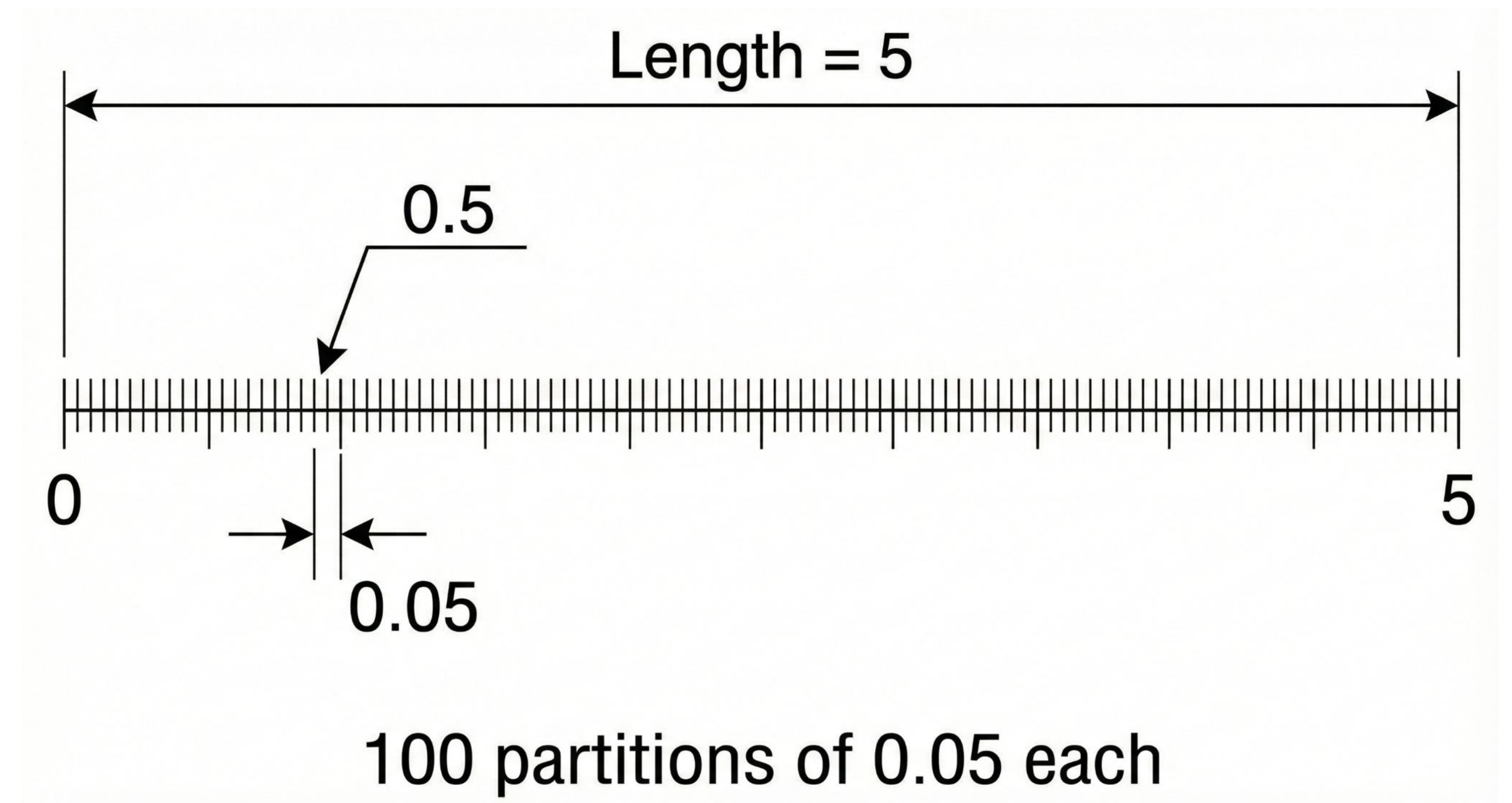
Finite Difference Formula for variable diffusion with reaction (Tridiagonal Form)

$$\alpha_i C_{i-1} + \beta_i C_i + \gamma_i C_{i+1} = 0$$

Where the coefficients are defined as:

Grid Generation

- Total Length $L = 5$ cm
- Step Size $\Delta x = 0.05$ cm
- Total Nodes $N = 101$
- Interior Points = 99 (Unknowns)



The Linear System

The FDM discretization results in a tridiagonal matrix $\mathbf{Ax} = \mathbf{b}$.

For 99 interior points, we solve for C1 to C99.

Note: Boundary terms are moved to the RHS vector.

$$\begin{bmatrix} -674.1 & 337.5 & 0 & \dots \\ 337.5 & -676.1 & 338.5 & \dots \\ 0 & 338.5 & -678.1 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} C_1 \\ C_2 \\ C_3 \\ \vdots \end{bmatrix} = \begin{bmatrix} -33650 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$$

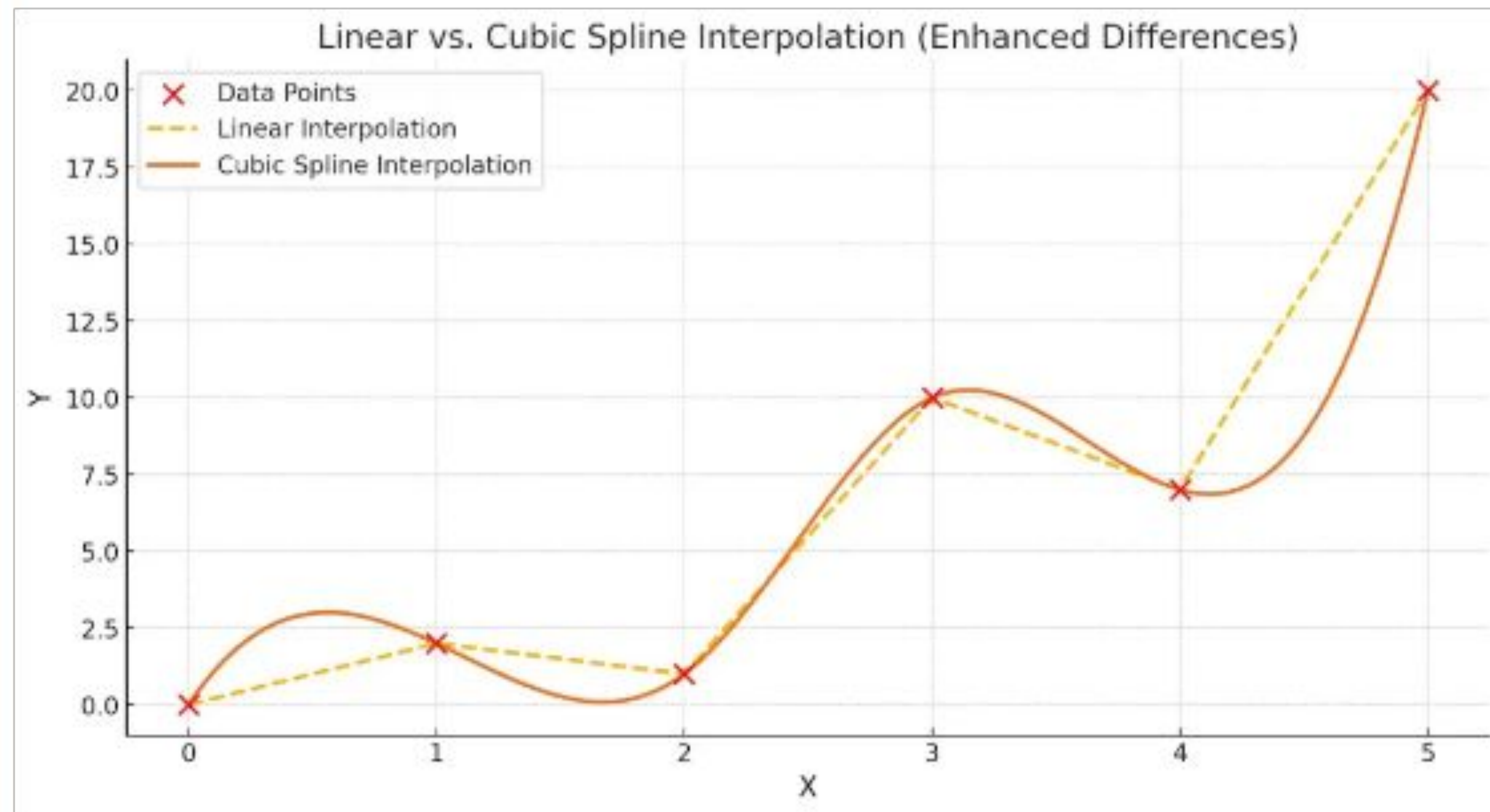
Finite difference method code

```
%% Finite Difference Method (using gauss elimination with partial pivoting) -----
N_in = N-2; %consider only interior points
A = zeros(N_in, N_in); % matrix of alpha,beta and gamma
b = zeros(N_in, 1); % matrix for the rhs
for r = 1:N_in
    i=r+1; % doing only on interior points
    Di = D_x(i);
    alpha = (Di / dx^2) - (slope/(2*dx));
    beta = (-2*Di/dx^2) - k_rate;
    gamma = (Di/dx^2) + (slope/(2*dx));
    A(r, r) = beta;
    if r > 1;
        A(r, r-1) = alpha;
    end
    if r < N_in;
        A(r, r+1) = gamma;
    end
    if r == 1; % boundary condition for x=0
        b(r) = -alpha * CA0;
    elseif r == N_in; %boundary condition for x=n
        b(r) = -gamma * CAn;
    end
end
end
```

gauss elimination

```
% Solve Ax = b using partial pivoting gauss elimination
B=[A, b]; % augmented matrix for gauss elimination forward elimination
n_sys = N_in;
for k = 1:N_in-1
    [~,p] = max(abs(B(k:N_in, k))); % partial pivoting
    p_row = p + k - 1;
    if p_row ~= k;
        temp = B(k, :); B(k, :) = B(p_row, :); B(p_row, :) = temp;
    end
    for i = k+1:N_in
        factor = B(i, k) / B(k, k);
        B(i, k:N_in+1) = B(i, k:N_in+1) - factor * B(k, k:N_in+1); % forward elimination
    end
end
C_in = zeros(N_in, 1); % creating concn matrix for interior pts
C_in(N_in) = B(N_in, N_in+1) / B(N_in,N_in);
for i = N_in-1:-1:1
    sum_val = sum(B(i, i+1:N_in) .* C_in(i+1:N_in)');
    C_in(i) = (B(i,N_in+1) - sum_val) / B(i, i);
end
C_augmented = [CA0; C_in; CAn];%creating augmented matrix of concentrations
fprintf('2.numerical data is ready. range: [%.2f, %.2f]\n', min(C_augmented), max(C_augmented))
```

Interpolation Technique



Natural Cubic Spline

While FDM gives us values at discrete points (0.05, 0.10...), we often need to know the concentration *between* these points.

Cubic Spline Interpolation fits a smooth third-order polynomial between each pair of nodes, ensuring continuous curvature across the entire domain, unlike linear interpolation which results in jagged edges.

Natural Cubic Spline: Formula Reference Sheet

1. Definitions & Model

Data Points: $(x_0, y_0), \dots, (x_n, y_n)$

Step Size: $h_i = x_{i+1} - x_i$

Step Size: $h_i = x_{i+1} - x_i$

Moments (Unknowns): $M_i = S''(x_i)$

Piecewise Cubic: $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$ for $x \in [x_i, x_{i+1}]$

2. Governing Linear System (for M_i)

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left[\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right] \text{ for } i = 1 \dots n - 1$$

Boundary Conditions (Natural) $M_0 = 0$ $M_n = 0$

3. Coefficient Formulas (after solving for M)

$a_i = y_i$	$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2M_i + M_{i+1})}{6}$	
$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2M_i + M_{i+1})}{6}$	$c_i = \frac{M_i}{2}$	$d_i = \frac{M_{i+1} - M_i}{6h_i}$

```

%% spline curves calculations
n_in = N - 2;
S = zeros(n_in, n_in); Y = zeros(n_in, 1);
for i = 1:n_in
    idx = i + 1;
    S(i, i) = 4;
    if i > 1; S(i, i-1) = 1; end;
    if i < n_in; S(i, i+1) = 1; end;
    Y(i) = (6 / dx^2) * (C_augmented(idx-1) - 2*C_augmented(idx) + C_augmented(idx+1));
end
augmentedS = [S, Y];
% guass elimination
for k=1:n_in-1
    [~, p] = max(abs(augmentedS(k:n_in, k))); %pivoting
    p_row=p+k-1;
    if p_row ~= k;
        temp = augmentedS(k, :);
        augmentedS(k, :) = augmentedS(p_row, :);
        augmentedS(p_row, :) = temp;
    end
    for i = k+1:n_in
        factor = augmentedS(i, k) / augmentedS(k, k);

```

spline curve code

```

        augmentedS(i, k:n_in+1) = augmentedS(i, k:n_in+1) - factor * augmentedS(k, k:n_in+1);
    end
end
M_in = zeros(n_in, 1); % matrix for the interior points M calculation
M_in(n_in) = augmentedS(n_in, n_in+1) / augmentedS(n_in, n_in);
for i = n_in-1:-1:1
    sum_val = sum(augmentedS(i, i+1:n_in) .* M_in(i+1:n_in)');
    M_in(i) = (augmentedS(i, n_in+1) - sum_val) / augmentedS(i, i);
end
M_augmented = [0; M_in; 0];
x_values = linspace(0, L, 200)';
C_spline = zeros(size(x_values));
for k = 1:length(x_values)
    xk = x_values(k);
    idx = floor(xk / dx)+1;
    if idx >= N;
        idx = N - 1;
    end
    h = dx;
    xi = x(idx);
    xi1 = x(idx+1);
    AA = (xi1 - xk)/h;
    BB = (xk - xi)/h;
    C_spline(k) = AA*C_augmented(idx) + BB*C_augmented(idx+1) + ((AA^3 - AA)*M_augmented(idx) + (BB^3 - BB)*M_augmented(idx+1));
end

```

$$\frac{d}{dx} \left(D(x) \frac{dc}{dx} \right) - KC = 0, \quad 0 < x < L$$

$$L = 5 \text{ cm}$$

$$D = 0.84 \text{ cm}^2/\text{day}, \quad x = 0$$

$$\text{BCs: } C_A = 100 \text{ mol/L} \quad x = 0$$

$$D = 1.09 \text{ cm}^2/\text{day}, \quad x = L$$

$$C_A = 100 \text{ mol/L} \quad x = L$$

$$D(x) = 0.84 + 0.05x \text{ (cm}^2/\text{day)} \quad K = 0.1 \text{ day}^{-1}$$

$$D(x) = \alpha + \beta x, \quad \alpha = 0.84 \\ \beta = 0.05$$

Expanding ODE:

$$D \frac{d^2 C_A}{dx^2} + \frac{dD}{dx} \frac{dC_A}{dx} - KC_A = 0$$

$$D(x) \frac{d^2 C_A}{dx^2} + \beta \frac{dC_A}{dx} - KC = 0$$

$$C'' + \frac{\beta}{\alpha + \beta x} C' - \frac{K}{\alpha + \beta x} C = 0$$

Transform the ODE:

$$z(x) = \frac{2}{\beta} \sqrt{K(\alpha + \beta x)}$$

Then

$$\frac{d^2 C}{dz^2} + \frac{1}{z} \frac{dC}{dz} - C = 0$$

$$C'' + \frac{1}{z} C' - C = 0 \rightarrow \text{modified Bessel eqn of order zero}$$

general soln.

$$C(x) = A I_0(z(x)) + B K_0(z(x))$$

$$I_0, K_0 \rightarrow \text{modified Bessel fns.}$$

$$z(x) = 40 \sqrt{0.084 + 0.005x}$$

At the boundaries

$$x = 0, \quad z_0 = 11.5931$$

$$x = 5, \quad z_L = 13.206$$

Applying BCs

$$C(0) = 100$$

$$C(L) = 100$$

$$\text{At } z_0 = 11.5931,$$

$$I_0(z_0) = 12828.1, \quad K_0(z_0) = 3.44 \times 10^{-6}$$

$$z_L = 13.206$$

$$I_0(z_L) = 26775.6, \quad K_0(z_L) = 1.82 \times 10^{-6}$$

$$C(0) = A I_0(z_0) + B K_0(z_0) = 100$$

$$C(L) = A I_0(z_L) + B K_0(z_L) = 100$$

$$A = 4.029174 \times 10^{-3}, \quad B = 1.435568 \times 10^{-7}$$

Final Analytic soln.

$$C(x) = 4.029174 \times 10^{-3} I_0(40 \sqrt{0.084 + 0.005x}) \\ + 1.435568 \times 10^{-7} K_0(40 \sqrt{0.084 + 0.005x})$$

analytical solution by hand

analytical solution using approximation for bessell function

```
Io_approx = @(z) (exp(z) ./ sqrt(2*pi*z)) .* (1 + 1./(8*z));
Ko_approx = @(z) (sqrt(pi./(2*z)) .* exp(-z)) .* (1 - 1./(8*z));

z_func = @(v) (2/slope) * sqrt(k_rate * (slope*v + D0));

% Using the mini functions instead of built-in bessell
val_i0_0 = Io_approx(z_func(0)); val_k0_0 = Ko_approx(z_func(0));
val_i0_L = Io_approx(z_func(L)); val_k0_L = Ko_approx(z_func(L));

det_val = val_i0_0 * val_k0_L - val_k0_0 * val_i0_L;
c1 = (100 * val_k0_L - 100 * val_k0_0) / det_val;
c2 = (100 * val_i0_0 - 100 * val_i0_L) / det_val;

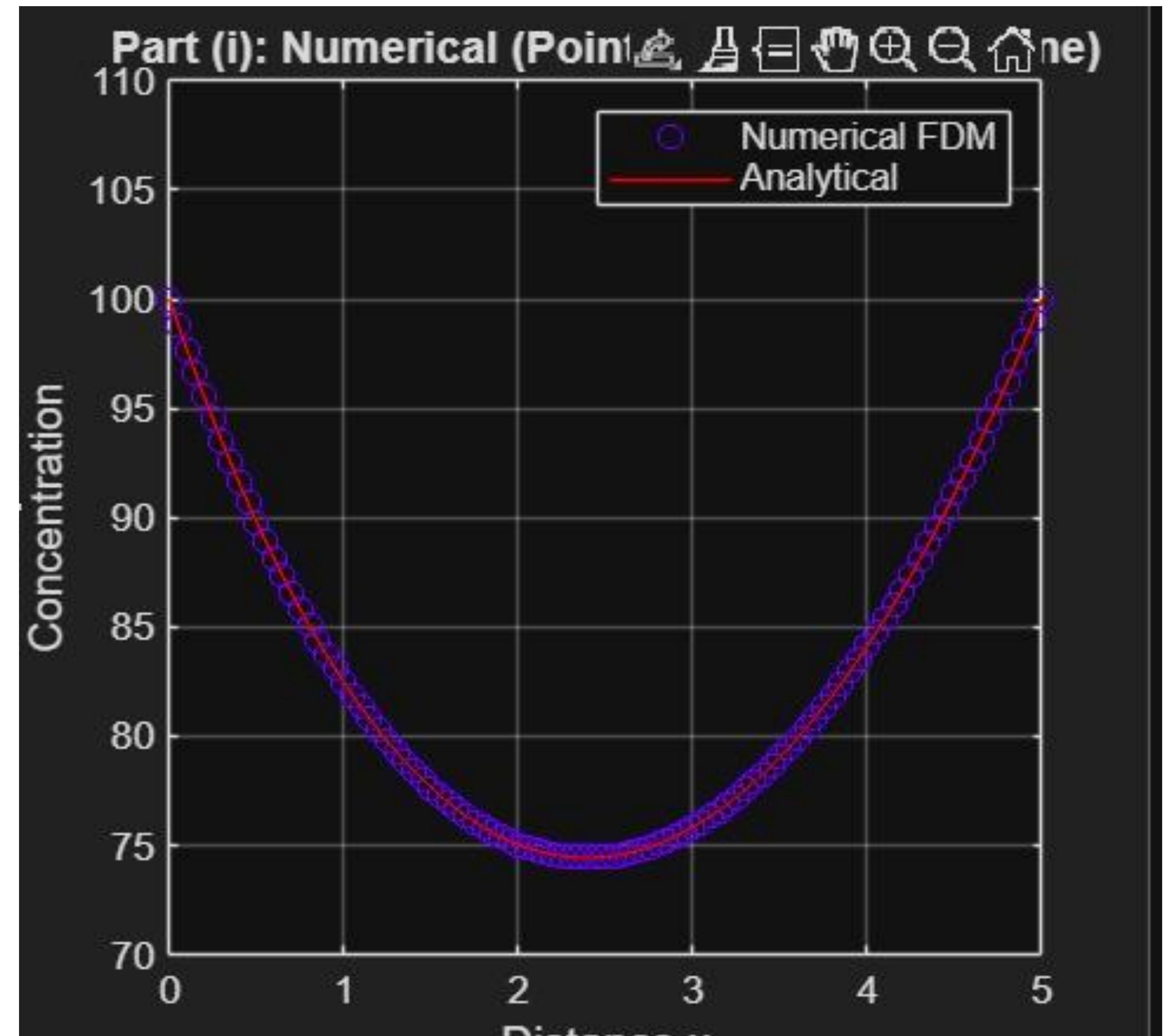
C_anal = c1 * Io_approx(z_func(x_values)) + c2 * Ko_approx(z_func(x_values));

% SAFETY CHECK: If C_anal has complex numbers, plot will fail. We take REAL part only.
if ~isreal(C_anal)
    fprintf('WARNING: Analytical solution has complex parts. Taking real part only.\n');
    C_anal = real(C_anal);
end
fprintf('3. All Data Calculated successfully.\n');
```

Visual Comparison

The plot visualizes the concentration drop in the center of the liquid layer due to the reaction consuming species A.

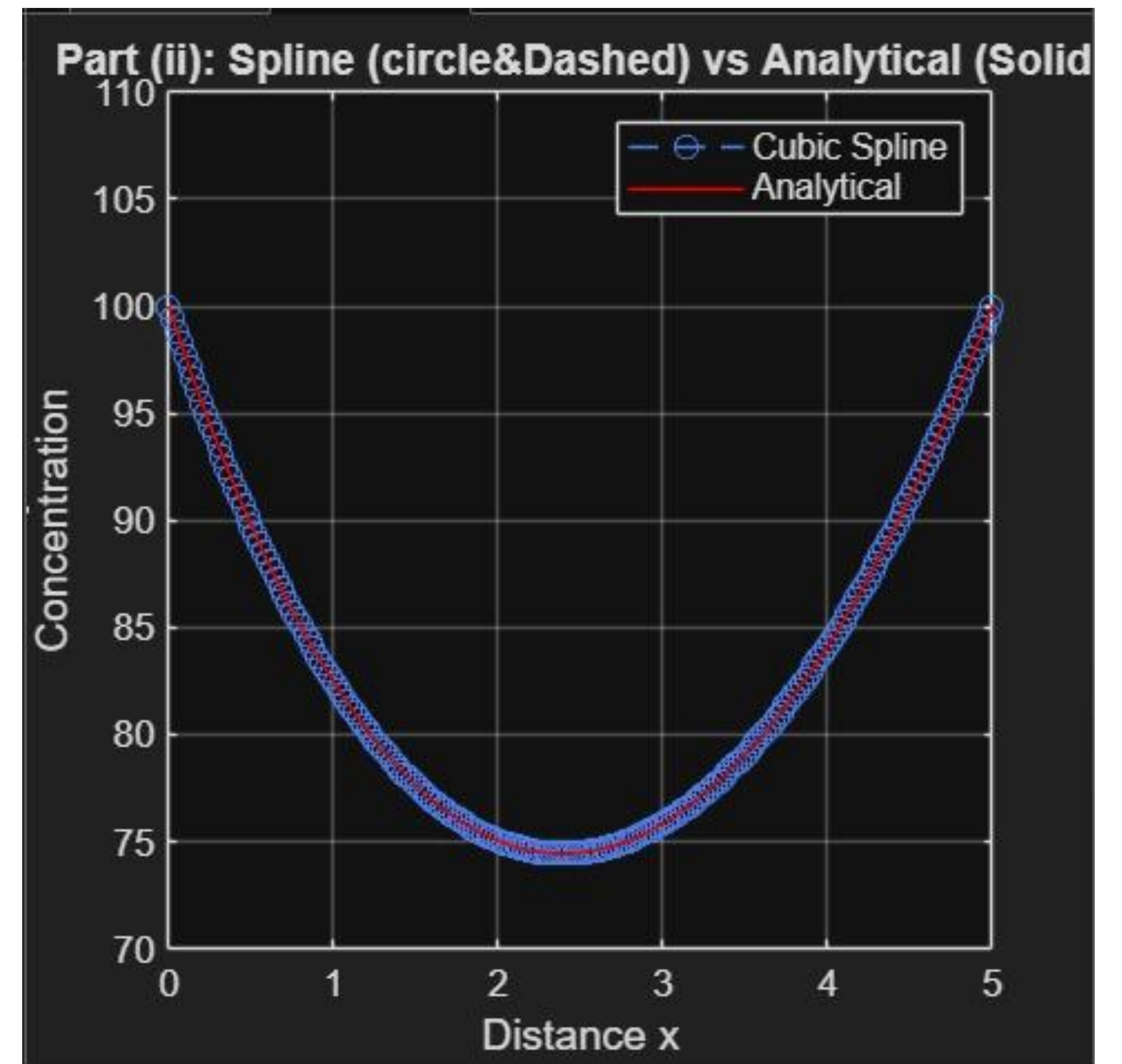
The **FDM points** (dots) align perfectly with the **Analytical Solution** (line), demonstrating the method's stability.



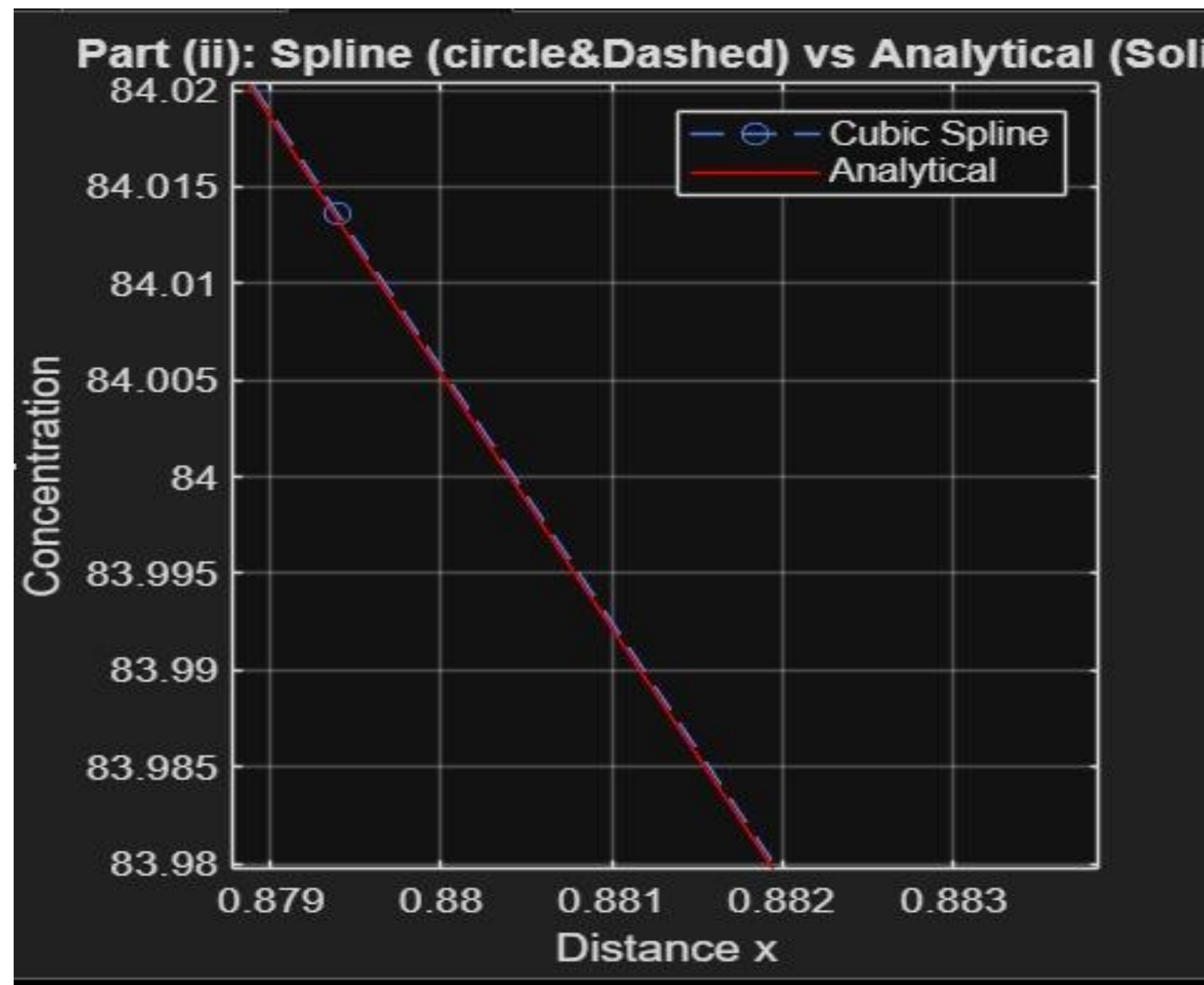
Visual Comparison

The plot visualizes the concentration drop in the center of the liquid layer due to the reaction consuming species A.

The **Cubic spline plot** (dots) align perfectly with the **Analytical Solution** (line), demonstrating the method's stability.



zoomed version spline vs analytical curve



Summary



Accurate

The Finite Difference Method successfully solved the variable-coefficient ODE with high precision.



Continuous

Natural Cubic Splines provided a smooth interpolation function for determining concentration at any arbitrary point.



Efficient

The tridiagonal matrix formulation ensures the solution is computationally inexpensive ($O(N)$ complexity).